

CAHIER DE CHARGES

Conception et Développement d'une Application de Gestion de Point de Vente (POS) pour les Débits de Boissons

Réalisé par

Étudiants en Génie Informatique des Systèmes Logistiques Intelligents (GSI)

Niveau : Licence 4

Technologies principales

Flutter – Django - PostgreSQL - API REST - Git & GitHub

Sous la supervision du :

Pr ATSA ETOUNDI Roger (Professeur Titulaire - Université de Yaoundé 1)

Juillet 2026

TABLE DES MATIÈRES

Page de garde

Table des matières

1. Introduction

- 1.1. Contexte du projet
- 1.2. Problématique
- 1.3. Justification du projet
- 1.4. Objectifs du cahier des charges
- 1.5. Méthodologie de réalisation

2. Présentation générale du projet

- 2.1. Présentation du projet
- 2.2. Vision du projet
- 2.3. Public cible
- 2.4. Domaine d'application
- 2.5. Périmètre du projet

3. Étude de l'existant

- 3.1. Fonctionnement actuel des débits de boissons
- 3.2. Analyse des solutions existantes
- 3.3. Limites des solutions actuelles
- 3.4. Solution proposée

4. Analyse des besoins

- 4.1. Identification des acteurs
- 4.2. Besoins fonctionnels
- 4.3. Besoins non fonctionnels
- 4.4. Contraintes du projet

5. Spécifications fonctionnelles

- 5.1. Module d'authentification
- 5.2. Gestion des utilisateurs
- 5.3. Gestion des catégories de produits
- 5.4. Gestion des produits (boissons et autres articles)
- 5.5. Gestion des stocks
- 5.6. Gestion des fournisseurs
- 5.7. Gestion des clients

- 5.8. Gestion des ventes (POS)
- 5.9. Gestion des paiements
- 5.10. Impression des reçus et factures
- 5.11. Gestion des dépenses
- 5.12. Tableau de bord
- 5.13. Rapports et statistiques
- 5.14. Paramètres de l'application

6. Architecture du système

- 6.1. Architecture générale
- 6.2. Architecture logicielle
- 6.3. Architecture matérielle
- 6.4. Architecture réseau

7. Choix technologiques

- 7.1. Technologies du frontend
- 7.2. Technologies du backend
- 7.3. Base de données
- 7.4. API et communication
- 7.5. Outils de développement

8. Modélisation UML

- 8.1. Diagramme de cas d'utilisation
- 8.2. Diagramme de classes
- 8.3. Diagrammes de séquence
- 8.4. Diagrammes d'activités
- 8.5. Diagramme d'états
- 8.6. Diagramme de composants
- 8.7. Diagramme de déploiement

9. Conception de la base de données

- 9.1. Dictionnaire des données
- 9.2. Modèle Conceptuel de Données (MCD)
- 9.3. Modèle Logique de Données (MLD)
- 9.4. Schéma relationnel

10. Conception des interfaces utilisateur

- 10.1. Principes de conception UI/UX
- 10.2. Écran de connexion
- 10.3. Tableau de bord
- 10.4. Interface de vente (POS)
- 10.5. Gestion des produits
- 10.6. Gestion des stocks
- 10.7. Gestion des utilisateurs
- 10.8. Rapports et statistiques
- 10.9. Paramètres

11. Sécurité du système

- 11.1. Authentification et autorisation
- 11.2. Gestion des rôles et permissions
- 11.3. Protection des données
- 11.4. Sauvegarde et restauration
- 11.5. Journalisation des activités

12. Planification du projet

- 12.1. Organisation des travaux
- 12.2. Phases de développement

13. Stratégie de tests

- 13.1. Tests unitaires
- 13.2. Tests d'intégration
- 13.3. Tests fonctionnels
- 13.4. Tests utilisateurs
- 13.5. Validation du système

14. Livrables du projet

15. Conclusion

16. Annexes

- 16.1. Captures d'écran et maquettes
- 16.2. Diagrammes complémentaires

CHAPITRE 1 : INTRODUCTION

1.1 Contexte du projet

La transformation numérique touche aujourd'hui tous les secteurs d'activité, y compris le commerce de détail et la restauration. Les débits de boissons, qu'il s'agisse de bars, de buvettes, de snacks, de caves ou de restaurants, sont confrontés à une gestion quotidienne de plus en plus complexe. Chaque jour, ces établissements réalisent un grand nombre d'opérations telles que la vente de boissons, la gestion des stocks, le suivi des approvisionnements, l'encaissement des paiements, la gestion des clients et la production de rapports financiers.

Dans de nombreux établissements, notamment les petites et moyennes entreprises, ces opérations sont encore effectuées manuellement ou à l'aide d'outils informatiques peu adaptés, comme des cahiers de caisse ou des feuilles de calcul. Cette méthode de gestion entraîne souvent des erreurs de saisie, des pertes financières, des difficultés de suivi des stocks, une faible traçabilité des opérations et une prise de décision moins efficace.

L'évolution des technologies de l'information offre aujourd'hui la possibilité de moderniser ces processus grâce à des solutions numériques performantes. Les systèmes de Point de Vente (Point of Sale – POS) permettent d'automatiser les opérations commerciales, d'améliorer la productivité du personnel et de fournir des informations fiables en temps réel pour une meilleure gestion de l'établissement.

C'est dans cette perspective que s'inscrit ce projet, dont l'objectif est de concevoir et de développer une application moderne de gestion de Point de Vente destinée aux débits de boissons. Cette application intégrera les principales fonctionnalités nécessaires à la gestion quotidienne d'un établissement tout en offrant une interface ergonomique, sécurisée et adaptée aux besoins des différents utilisateurs.

1.2 Problématique

Les débits de boissons doivent assurer simultanément la gestion des ventes, des stocks, des approvisionnements, des paiements, des utilisateurs et des statistiques commerciales. Lorsque ces opérations sont réalisées manuellement ou avec des outils inadaptés, plusieurs difficultés apparaissent.

Parmi les problèmes les plus fréquents figurent les erreurs d'encaissement, les écarts de caisse, les ruptures de stock imprévues, les difficultés à suivre les mouvements des produits, la perte d'informations importantes ainsi que l'absence de tableaux de bord permettant d'évaluer les performances de l'établissement.

Par ailleurs, les responsables disposent rarement d'informations fiables et actualisées pour analyser les ventes, identifier les produits les plus demandés ou anticiper les besoins en approvisionnement. Cette situation peut entraîner une baisse de rentabilité et compliquer la prise de décisions stratégiques.

Face à ces constats, il devient nécessaire de mettre en place une solution informatique capable de centraliser les données, d'automatiser les processus de gestion et de fournir des indicateurs fiables afin d'améliorer l'efficacité opérationnelle des débits de boissons.

La problématique principale de ce projet peut ainsi être formulée de la manière suivante :

Comment concevoir et développer une application de gestion de Point de Vente performante, intuitive et sécurisée permettant d'optimiser la gestion des ventes, des stocks, des utilisateurs et des opérations financières au sein d'un débit de boissons ?

1.3 Justification du projet

La réalisation de cette application répond à un besoin réel exprimé par de nombreux gestionnaires de débits de boissons souhaitant améliorer leur organisation et réduire les erreurs liées à une gestion essentiellement manuelle.

L'automatisation des processus de vente permettra de réduire le temps de traitement des opérations tout en améliorant la précision des transactions. La centralisation des informations facilitera le suivi des activités commerciales et la consultation des données historiques.

L'application contribuera également à une meilleure gestion des stocks grâce au suivi automatique des entrées et des sorties de produits, à la génération d'alertes en cas de stock faible et à la consultation permanente des quantités disponibles.

Du point de vue décisionnel, les tableaux de bord et les rapports statistiques offriront aux responsables une vision globale de l'activité commerciale, facilitant ainsi l'évaluation des performances, le suivi du chiffre d'affaires et la planification des approvisionnements.

Enfin, ce projet constitue une opportunité de mettre en pratique les connaissances acquises dans les domaines du génie logiciel, de la conception des systèmes d'information, de la modélisation UML, du développement d'applications multiplateformes avec Flutter et de la conception de bases de données relationnelles.

1.4 Objectifs du cahier des charges

Objectif général

Le présent cahier des charges a pour objectif de définir avec précision les besoins fonctionnels et techniques nécessaires à la conception et au développement d'une application moderne de gestion de Point de Vente destinée aux débits de boissons.

Objectifs spécifiques

De manière spécifique, ce cahier des charges vise à :

- décrire le contexte et les enjeux du projet ;
- identifier les différents acteurs du système et leurs responsabilités ;
- recenser les besoins fonctionnels et non fonctionnels de l'application ;
- définir les contraintes techniques, matérielles et organisationnelles du projet ;
- présenter les fonctionnalités attendues de chaque module de l'application ;
- préciser l'architecture générale du système ;
- servir de référence tout au long des phases de conception, de développement, de test et de déploiement ;
- faciliter la communication entre les différentes parties prenantes du projet ;
- garantir que la solution développée réponde efficacement aux besoins des futurs utilisateurs.

1.5 Méthodologie de réalisation

La réalisation de ce projet suivra une démarche structurée reposant sur les principes du génie logiciel.

Dans un premier temps, une phase d'analyse permettra d'étudier le fonctionnement des débits de boissons, d'identifier les besoins des utilisateurs et de définir les exigences fonctionnelles et non fonctionnelles du futur système.

La phase de conception consistera ensuite à modéliser l'application à l'aide du langage UML. Les différents diagrammes (cas d'utilisation, classes, séquences, activités) permettront de représenter le comportement et l'architecture du système avant son développement.

Le développement de l'application se réalisera selon une architecture client-serveur. L'interface utilisateur sera développée avec Flutter afin de garantir une compatibilité multiplateforme, tandis que le backend reposera sur une API REST assurant la communication avec une base de données PostgreSQL.

Après le développement, des tests unitaires, des tests d'intégration et des tests fonctionnels seront réalisés afin de vérifier la conformité de l'application avec les exigences définies dans le présent cahier des charges. Une phase de validation permettra enfin de s'assurer que la solution répond pleinement aux besoins des utilisateurs avant son déploiement.

CHAPITRE 2 : PRÉSENTATION GÉNÉRALE DU PROJET

2.1 Présentation du projet

Le projet consiste en la conception et le développement d'une application de gestion de Point de Vente (Point of Sale – POS) destinée aux débits de boissons. Cette application a pour vocation de numériser et d'automatiser les principales activités de gestion d'un établissement commercial afin d'améliorer son efficacité opérationnelle, de réduire les erreurs liées aux traitements manuels et de faciliter la prise de décision.

L'application permettra d'assurer la gestion complète des ventes, des produits, des stocks, des fournisseurs, des clients, des utilisateurs ainsi que la génération de rapports et de statistiques en temps réel. Elle offrira une interface ergonomique, intuitive et sécurisée, adaptée aussi bien aux petites structures qu'aux établissements de grande capacité.

Conçue selon une architecture client-serveur, l'application reposera sur un frontend développé avec Flutter, un backend exposant une API REST et une base de données PostgreSQL. Cette architecture garantira une bonne évolutivité, une maintenance facilitée et une compatibilité avec différents supports (ordinateur, tablette et smartphone).

Le système sera organisé autour de plusieurs modules fonctionnels indépendants mais interconnectés, permettant d'assurer une gestion centralisée de toutes les opérations réalisées au sein du débit de boissons.

2.2 Vision du projet

La vision de ce projet est de proposer une solution moderne, fiable et évolutive permettant aux responsables de débits de boissons de disposer d'un véritable outil d'aide à la gestion.

À travers cette application, il s'agit de remplacer les méthodes traditionnelles de gestion par un système numérique capable d'automatiser les opérations commerciales, de sécuriser les transactions et de fournir des informations fiables pour la prise de décision.

À moyen terme, l'application pourra intégrer des fonctionnalités avancées telles que la gestion multi-établissements, la synchronisation dans le cloud, les tableaux de bord décisionnels, les programmes de fidélisation des clients, les commandes en ligne, les paiements électroniques, les notifications intelligentes et l'analyse prédictive des ventes.

Cette vision vise à faire évoluer l'application vers une plateforme complète de gestion commerciale capable d'accompagner la croissance des établissements utilisant le système.

2.3 Public cible

L'application est destinée aux différents acteurs intervenant dans la gestion quotidienne des débits de boissons. Elle pourra être utilisée par :

- les propriétaires de bars, snacks, buvettes et caves à boissons ;
- les gérants d'établissements ;
- les caissiers chargés des opérations de vente et d'encaissement ;
- les magasiniers responsables de la gestion des stocks ;
- les serveurs amenés à enregistrer les commandes ;
- les comptables assurant le suivi financier de l'activité ;
- les administrateurs du système chargés de la configuration, de la sécurité et de la gestion des utilisateurs.

Grâce à une gestion des rôles et des permissions, chaque utilisateur accédera uniquement aux fonctionnalités correspondant à ses responsabilités.

2.4 Domaine d'application

Le projet s'inscrit dans le domaine des systèmes d'information de gestion et plus précisément dans celui des solutions de Point de Vente (POS).

L'application sera adaptée à différents types d'établissements commerciaux, notamment :

- les bars ;
- les buvettes ;
- les snacks ;
- les caves à boissons ;
- les restaurants proposant un service de boissons ;
- les hôtels disposant d'un bar ;
- les espaces de loisirs et de divertissement.

Elle contribuera à l'automatisation des activités commerciales, à l'amélioration de la gestion des stocks et à la production d'indicateurs permettant d'optimiser les performances de l'établissement.

2.5 Périmètre du projet

Le périmètre du projet définit les fonctionnalités qui seront prises en charge par l'application ainsi que les limites de la solution développée.

Fonctionnalités incluses

L'application prendra en charge :

- l'authentification et la gestion sécurisée des utilisateurs ;
- la gestion des rôles et des permissions ;
- la gestion des catégories de produits ;
- la gestion des boissons et autres articles commercialisés ;
- la gestion des stocks et des inventaires ;
- la gestion des fournisseurs ;
- la gestion des clients ;
- l'enregistrement des ventes ;
- la gestion des paiements (espèces, Mobile Money, carte bancaire et paiement mixte) ;
- l'impression des reçus de caisse ;
- la gestion des dépenses courantes ;
- le suivi des mouvements de stock ;
- la génération de rapports de ventes et de statistiques ;
- la consultation d'un tableau de bord décisionnel ;
- la sauvegarde des données et la journalisation des opérations.

Fonctionnalités non couvertes dans cette première version

La première version de l'application ne prendra pas en charge :

- la vente en ligne ;
- les commandes via une application mobile client ;
- l'intégration avec des plateformes de livraison ;

- la comptabilité générale complète ;
- la gestion des ressources humaines et de la paie ;
- l'analyse prédictive basée sur l'intelligence artificielle ;
- la gestion de plusieurs établissements à partir d'une seule installation.

Ces fonctionnalités pourront être intégrées dans les versions futures du système afin d'accompagner son évolution et de répondre à de nouveaux besoins.

CHAPITRE 3 : ANALYSE DU CONTEXTE ET ÉTUDE DE L'EXISTANT

L'analyse du contexte et de l'existant constitue une étape fondamentale dans la réalisation d'un projet informatique. Elle permet de comprendre l'environnement dans lequel le système sera déployé, d'identifier les besoins réels des utilisateurs, et de mettre en évidence les insuffisances des solutions actuellement utilisées.

Dans le cadre de ce projet de conception d'une application de gestion de Point de Vente (POS) pour les débits de boissons, cette analyse vise à étudier les méthodes traditionnelles de gestion ainsi que les solutions numériques existantes afin de mieux justifier la mise en place de notre système.

3.2 Contexte général

Les débits de boissons (bars, snacks, buvettes, restaurants, caves à boissons) jouent un rôle important dans l'économie locale. Leur gestion repose sur plusieurs activités quotidiennes telles que :

- la vente des produits ;
- la gestion des stocks ;
- les achats auprès des fournisseurs ;
- la gestion des paiements ;
- le suivi des dépenses ;
- la relation avec les clients.

Dans de nombreux établissements, ces opérations sont encore réalisées de manière manuelle ou semi-informatisée, ce qui entraîne souvent des erreurs, des pertes financières et un manque de visibilité sur la performance globale de l'activité.

3.3 Analyse de l'existant

3.3.1 Gestion manuelle

Dans plusieurs petits établissements, la gestion est encore effectuée à l'aide de cahiers ou de feuilles Excel simples.

Avantages

- Coût très faible ;
- Simplicité d'utilisation ;
- Pas besoin d'équipement informatique avancé.

Inconvénients

- Risque élevé d'erreurs de calcul ;
- Perte ou détérioration des données ;
- Absence de centralisation des informations ;

- Difficulté à produire des rapports fiables ;
- Aucun suivi en temps réel des stocks ;
- Manque de sécurité des données.

3.3.2 Solutions logicielles existantes

Il existe déjà plusieurs solutions de Point of Sale sur le marché, telles que :

- Loyverse POS ;
- Square POS ;
- Odoo POS ;
- Lightspeed ;
- Vend POS.

Avantages

- Automatisation des ventes ;
- Gestion des stocks ;
- Génération de rapports ;
- Interface moderne ;
- Accès multi-utilisateurs.

Limites dans le contexte local

Malgré leurs performances, ces solutions présentent certaines limites pour les débits de boissons dans notre contexte :

- coût élevé de certaines licences ;
- dépendance à une connexion Internet stable ;
- complexité d'utilisation pour certains utilisateurs ;
- manque de personnalisation selon les réalités locales ;
- intégration limitée avec les moyens de paiement locaux (Mobile Money, etc.) ;
- adaptation insuffisante aux habitudes de gestion des petits établissements.

3.4 Problématique

À partir de l'analyse de l'existant, plusieurs problèmes majeurs sont identifiés :

- difficulté de suivi précis des ventes et des stocks ;
- pertes financières dues aux erreurs humaines ;
- absence de système centralisé de gestion ;
- manque de transparence dans les opérations ;

- difficulté à obtenir des rapports fiables en temps réel ;
- gestion inefficace des fournisseurs et des approvisionnements.

Ainsi, la problématique principale peut être formulée comme suit :

Comment concevoir une application de gestion de Point de Vente simple, efficace, sécurisée et adaptée aux besoins réels des débits de boissons ?

3.5 Solution proposée

Pour répondre à ces insuffisances, il est proposé de concevoir une application POS moderne offrant :

- une gestion centralisée des ventes, stocks et achats ;
- une interface simple et intuitive adaptée aux utilisateurs locaux ;
- un suivi en temps réel des opérations ;
- une gestion sécurisée des utilisateurs et des accès ;
- une génération automatique de rapports ;
- une compatibilité avec les besoins des débits de boissons ;
- une possibilité d'évolution vers une solution plus complète.

3.6 Objectifs de l'étude

Cette étude vise à :

- analyser les méthodes de gestion existantes ;
- identifier les insuffisances des solutions actuelles ;
- définir les besoins réels des utilisateurs ;
- proposer une solution adaptée et évolutive ;
- justifier la mise en place du système proposé.

L'analyse du contexte et de l'existant met en évidence les limites des méthodes traditionnelles de gestion ainsi que les insuffisances de certaines solutions existantes dans le contexte local. Elle confirme la nécessité de développer une application POS adaptée aux réalités des débits de boissons, capable de centraliser les données, d'automatiser les opérations et de fournir des informations fiables pour la prise de décision.

Cette étude constitue ainsi une base essentielle pour la définition des spécifications fonctionnelles et la conception du système.

CHAPITRE 4 : ANALYSE DES BESOINS

L'analyse des besoins constitue une étape fondamentale dans la conception d'un système d'information. Elle permet d'identifier les attentes des futurs utilisateurs, de définir les fonctionnalités que devra offrir l'application et de préciser les exigences de qualité auxquelles le système devra répondre. Dans le cadre de ce projet, cette analyse porte sur les activités quotidiennes d'un débit de boissons afin de proposer une solution de gestion adaptée, performante et évolutive.

4.1 Identification des acteurs

Un acteur est une personne ou un système externe qui interagit avec l'application afin d'exécuter une ou plusieurs tâches. Chaque acteur dispose de responsabilités et de droits d'accès spécifiques.

4.1.1 Administrateur

L'administrateur est le principal responsable de la configuration du système. Il possède tous les privilèges et supervise l'ensemble des opérations de l'application.

Responsabilités

- créer, modifier et supprimer les comptes utilisateurs ;
- attribuer les rôles et les permissions ;
- configurer les paramètres de l'application ;
- consulter tous les rapports ;
- effectuer les sauvegardes et restaurations ;
- superviser la sécurité du système.

4.1.2 Gérant

Le gérant assure la gestion opérationnelle de l'établissement.

Responsabilités

- superviser les ventes ;
- suivre les performances commerciales ;
- gérer les produits ;
- consulter les statistiques ;
- gérer les fournisseurs ;
- contrôler les stocks ;

- autoriser certaines opérations sensibles (annulation d'une vente, remise exceptionnelle, etc.).

4.1.3 Caissier

Le caissier est chargé d'enregistrer les ventes et les paiements.

Responsabilités

- ouvrir une session de caisse ;
- enregistrer les commandes ;
- appliquer des remises autorisées ;
- encaisser les paiements ;
- imprimer les reçus ;
- clôturer la caisse en fin de service.

4.1.4 Magasinier (Logisticien)

Le magasinier est responsable de la gestion physique des stocks.

Responsabilités

- enregistrer les entrées de stock ;
- enregistrer les sorties exceptionnelles ;
- réaliser les inventaires ;
- signaler les ruptures de stock ;
- contrôler les quantités disponibles.

4.1.5 Serveur

Selon l'organisation de l'établissement, le serveur peut utiliser le système pour enregistrer les commandes des clients avant leur paiement.

Responsabilités

- créer une commande ;
- associer une commande à une table (si nécessaire) ;
- transmettre la commande au caissier ;
- consulter l'état des commandes.

4.1.6 Comptable

Le comptable exploite les données financières produites par l'application.

Responsabilités

- consulter les rapports financiers ;
- suivre les dépenses ;
- analyser le chiffre d'affaires ;
- exporter les données comptables.

4.2 Besoins fonctionnels

Les besoins fonctionnels représentent les services que le système devra fournir à ses utilisateurs.

Gestion des utilisateurs

Le système devra permettre :

- l'authentification sécurisée ;
- la création des comptes utilisateurs ;
- la modification des informations utilisateur ;
- La modification du mot de passe ;
- la désactivation d'un compte ;
- la gestion des rôles ;
- la gestion des permissions.

Gestion des produits

Le système devra permettre :

- l'ajout d'un produit ;
- la modification des informations d'un produit ;
- la suppression d'un produit ;
- la classification par catégories ;
- la gestion des prix ;
- la recherche rapide des produits ;
- l'affichage des stocks disponibles.

Gestion des catégories

Le système devra permettre :

- créer une catégorie ;
- modifier une catégorie ;
- supprimer une catégorie ;
- classer automatiquement les produits.

Gestion des fournisseurs

Le système devra permettre :

- enregistrer un fournisseur ;
- modifier ses informations ;
- consulter son historique ;
- gérer les livraisons.

Gestion des achats

Le système devra permettre :

- enregistrer les achats ;
- mettre automatiquement à jour les stocks ;
- consulter les historiques d'achats ;
- suivre les coûts d'approvisionnement.

Gestion des stocks

Le système devra permettre :

- enregistrer les entrées ;
- enregistrer les sorties ;
- suivre les mouvements ;
- effectuer des inventaires ;
- recevoir des alertes de rupture ;
- consulter les quantités en temps réel.

Gestion des ventes

Le système devra permettre :

- créer une vente ;
- ajouter ou supprimer des articles du panier ;
- modifier les quantités ;
- appliquer une remise autorisée ;
- annuler une vente selon les permissions ;
- calculer automatiquement le montant total ;
- enregistrer les paiements ;
- imprimer les reçus.

Gestion des paiements

Le système devra gérer :

- paiement en espèces ;
- paiement par Mobile Money ;
- paiement par carte bancaire ;
- paiement mixte.

Gestion des clients

Le système devra permettre :

- enregistrer un client ;
- consulter son historique d'achats ;
- gérer un programme de fidélité (évolution future).

Tableau de bord

Le système devra afficher :

- chiffre d'affaires journalier ;
- ventes du jour ;
- bénéfices estimés ;
- produits les plus vendus ;
- stock faible ;
- meilleures ventes.

Rapports

Le système devra produire :

- rapport journalier ;
- rapport hebdomadaire ;
- rapport mensuel ;
- rapport annuel ;
- rapport des ventes ;
- rapport des dépenses ;
- rapport des achats ;
- rapport des mouvements de stock.

4.3 Besoins non fonctionnels

En plus des fonctionnalités, le système devra satisfaire plusieurs exigences de qualité.

Performance

L'application devra répondre rapidement aux actions des utilisateurs et permettre l'enregistrement instantané des ventes.

Sécurité

Le système devra garantir :

- l'authentification des utilisateurs ;
- le chiffrement des mots de passe ;
- la gestion des rôles ;
- la protection des données sensibles ;
- la journalisation des actions importantes.

Disponibilité

L'application devra être disponible pendant toute la durée d'ouverture de l'établissement.

Fiabilité

Les données enregistrées devront être cohérentes, exactes et protégées contre toute perte accidentelle.

Ergonomie

L'interface devra être :

- simple ;
- intuitive ;
- moderne ;
- adaptée aux écrans tactiles ;
- facile à prendre en main.

Évolutivité

Le système devra pouvoir intégrer facilement de nouveaux modules sans remettre en cause son architecture.

Maintenabilité

L'application devra être conçue de manière modulaire afin de faciliter les corrections et les évolutions futures.

Portabilité

L'application devra fonctionner sur plusieurs plateformes compatibles avec Flutter et communiquer avec le backend via une API REST.

4.4 Contraintes du projet

La réalisation du projet devra respecter plusieurs contraintes.

Contraintes techniques

- Frontend développé avec Flutter.
- Backend basé sur une architecture REST.
- Base de données PostgreSQL.
- Utilisation d'un système de gestion de versions Git.
- Architecture client-serveur.

Contraintes matérielles

Le système devra fonctionner sur :

- ordinateur de bureau ;
- ordinateur portable ;

- tablette ;
- terminal tactile compatible.

Il devra également être compatible avec les périphériques suivants :

- imprimante thermique ;
- lecteur de codes-barres (si disponible) ;
- tiroir-caisse ;
- douchette code-barres.

Contraintes organisationnelles

Le projet devra :

- respecter le calendrier de développement ;
- répondre aux besoins des futurs utilisateurs ;
- produire une documentation complète ;
- garantir la qualité du code développé.

Contraintes réglementaires

L'application devra assurer :

- la confidentialité des informations ;
- la protection des données enregistrées ;
- la traçabilité des opérations effectuées ;
- la conservation des historiques de ventes et de paiements.

Cette analyse des besoins met en évidence les attentes fonctionnelles et techniques des différents utilisateurs du système. Les acteurs identifiés, les besoins exprimés ainsi que les contraintes recensées constituent les fondations de la conception de l'application POS.

Les exigences présentées dans ce chapitre serviront de référence pour la modélisation UML, la conception de la base de données, l'élaboration des interfaces utilisateur et le développement des différents modules fonctionnels qui seront détaillés dans le chapitre suivant consacré aux spécifications fonctionnelles.

CHAPITRE 5 : SPÉCIFICATIONS FONCTIONNELLES

Les spécifications fonctionnelles décrivent avec précision les fonctionnalités attendues de l'application de gestion de Point de Vente (POS). Elles définissent le comportement du système, les interactions entre les utilisateurs et les différents modules, ainsi que les règles de gestion qui devront être respectées lors du développement.

L'application sera organisée en plusieurs modules indépendants mais interconnectés, permettant une gestion centralisée des opérations commerciales d'un débit de boissons.

5.2 Module d'authentification

Objectif

Garantir un accès sécurisé à l'application.

Fonctionnalités

- Connexion avec identifiant et mot de passe.
- Déconnexion sécurisée.
- Changement de mot de passe.
- Réinitialisation du mot de passe par un administrateur.
- Gestion des sessions.
- Verrouillage automatique après plusieurs tentatives infructueuses.

Règles de gestion

- Chaque utilisateur possède un identifiant unique.
- Les mots de passe sont chiffrés.
- Les droits d'accès dépendent du rôle de l'utilisateur.
- Toute connexion est enregistrée dans un journal d'activité.

5.3 Module de gestion des utilisateurs

Objectif

Permettre l'administration des utilisateurs du système.

Fonctionnalités

- Ajouter un utilisateur.

- Modifier un utilisateur.
- Désactiver un utilisateur.
- Réactiver un utilisateur.
- Supprimer un utilisateur.
- Attribuer un rôle.
- Définir les permissions.
- Consulter la liste des utilisateurs.
- Rechercher un utilisateur.

Informations gérées

- Nom
- Prénom
- Téléphone
- Adresse
- Email
- Nom d'utilisateur
- Mot de passe
- Photo (optionnelle)
- Statut
- Date de création

5.4 Module de gestion des catégories

Objectif

Organiser les produits selon leur nature.

Fonctionnalités

- Ajouter une catégorie.
- Modifier une catégorie.
- Supprimer une catégorie.
- Activer ou désactiver une catégorie.
- Associer des produits à une catégorie.

Exemples :

- Bières
- Vins
- Whiskies
- Champagnes
- Jus
- Eaux minérales
- Boissons gazeuses
- Cocktails
- Spiritueux
- Snacks

5.5 Module de gestion des produits

Objectif

Administrer les boissons et autres produits commercialisés.

Informations enregistrées

- Code produit
- Code-barres
- Nom
- Catégorie
- Prix d'achat
- Prix de vente
- Marque
- Contenance
- Quantité disponible
- Stock minimum
- Description
- Photo

Fonctionnalités

- Ajouter un produit.
- Modifier un produit.
- Supprimer un produit.
- Consulter les stocks.
- Rechercher rapidement.
- Scanner un code-barres.
- Désactiver un produit.

5.6 Module de gestion des fournisseurs

Objectif

Gérer les partenaires fournissant les produits.

Fonctionnalités

- Ajouter un fournisseur.
- Modifier ses informations.
- Consulter son historique.
- Enregistrer une livraison.
- Suivre les commandes.
- Gérer les contacts.

5.7 Module de gestion des achats

Objectif

Enregistrer les approvisionnements.

Fonctionnalités

- Nouvelle commande.
- Réception d'une livraison.
- Mise à jour automatique du stock.
- Historique des achats.
- Annulation d'un achat.

- Génération d'un bon de réception.

5.8 Module de gestion des stocks

Objectif

Assurer le suivi permanent des quantités disponibles.

Fonctionnalités

- Entrées de stock.
- Sorties de stock.
- Inventaire.
- Ajustement manuel.
- Historique.
- Alertes de rupture.
- Alertes de stock faible.
- Valorisation du stock.

Règles de gestion

- Toute vente diminue automatiquement le stock.
- Toute livraison augmente automatiquement le stock.
- Les mouvements sont historisés.

5.9 Module de Point de Vente (POS)

Objectif

Enregistrer les ventes.

Fonctionnalités

- Ouverture de caisse.
- Fermeture de caisse.
- Création d'une vente.
- Panier dynamique.
- Recherche rapide.
- Scanner un produit.

- Calcul automatique.
- Gestion des quantités.
- Suppression d'un article.
- Mise en attente d'une vente.
- Reprise d'une vente.
- Annulation.
- Validation.
- Impression du reçu.

Informations affichées

- Produits
- Prix
- Quantités
- Sous-total
- TVA (si applicable)
- Remise
- Total
- Montant reçu
- Monnaie à rendre

5.10 Module de gestion des paiements

Objectif

Enregistrer les règlements.

Moyens de paiement

- Espèces
- Mobile Money
- Carte bancaire
- Paiement mixte

Fonctionnalités

- Validation du paiement.
- Calcul automatique de la monnaie.
- Historique.
- Impression du reçu.
- Annulation selon autorisation.

5.11 Module de gestion des clients

Objectif

Conserver les informations des clients réguliers.

Fonctionnalités

- Ajouter un client.
- Modifier ses informations.
- Historique d'achats.
- Statistiques.
- Programme de fidélité (version future).

5.12 Module de gestion des dépenses

Objectif

Enregistrer toutes les dépenses de fonctionnement.

Fonctionnalités

- Ajouter une dépense.
- Modifier une dépense.
- Catégoriser les dépenses.
- Historique.
- Rapports.

Exemples

- Eau
- Électricité

- Transport
- Salaire
- Entretien
- Fournitures

5.13 Module de rapports et statistiques

Objectif

Aider les responsables dans la prise de décision.

Rapports disponibles

- Ventes journalières.
- Ventes mensuelles.
- Ventes annuelles.
- Chiffre d'affaires.
- Produits les plus vendus.
- Produits les moins vendus.
- Dépenses.
- Stocks.
- Inventaires.
- Achats.
- Bénéfices.
- Performances des caissiers.

Exportation

- PDF
- Excel
- Impression directe

5.14 Module Tableau de bord

Le tableau de bord affichera en temps réel :

- chiffre d'affaires du jour ;

- ventes du jour ;
- nombre de tickets ;
- bénéfice estimé ;
- dépenses du jour ;
- produits en rupture ;
- alertes de stock ;
- meilleures ventes ;
- graphiques des ventes ;
- évolution du chiffre d'affaires.

5.15 Module Paramètres

Ce module permettra :

- configurer l'entreprise ;
- personnaliser le logo ;
- gérer les taxes ;
- configurer les imprimantes ;
- gérer les devises ;
- sauvegarder les données ;
- restaurer les sauvegardes ;
- configurer les notifications.

5.16 Notifications

Le système devra générer automatiquement des notifications lorsque :

- le stock devient faible ;
- un produit est en rupture ;
- une sauvegarde échoue ;
- une connexion suspecte est détectée ;
- une caisse n'est pas clôturée ;
- un produit arrive à expiration (si cette fonctionnalité est activée).

5.17 Journal des activités

Toutes les opérations importantes devront être enregistrées :

- connexions ;
- déconnexions ;
- ventes ;
- suppressions ;
- modifications ;
- annulations ;
- sauvegardes ;
- restaurations ;
- changements de paramètres.

5.18 Règles générales de gestion

Le système devra respecter les règles suivantes :

- Un utilisateur ne peut accéder qu'aux fonctionnalités autorisées par son rôle.
- Une vente validée met automatiquement à jour le stock.
- Une caisse doit être ouverte avant toute vente.
- Une caisse ne peut être clôturée qu'après vérification des encaissements.
- Un produit indisponible ne peut pas être vendu.
- Chaque vente possède un identifiant unique.
- Chaque mouvement de stock est historisé.
- Les données supprimées sont archivées lorsque cela est nécessaire afin d'assurer leur traçabilité.

Les spécifications fonctionnelles définissent précisément le comportement attendu de l'application POS. Elles décrivent les différents modules du système ainsi que leurs interactions, constituant ainsi une référence pour la modélisation UML, la conception de la base de données, le développement de l'application Flutter et la mise en œuvre des services backend.

Ces spécifications serviront également de base à la rédaction des scénarios de tests et à la validation finale de la solution.

CHAPITRE 6 : ARCHITECTURE DU SYSTÈME

L'architecture du système décrit l'organisation générale de l'application ainsi que les interactions entre ses différents composants. Elle définit la manière dont les données circulent entre les utilisateurs, les interfaces, les services métiers et la base de données.

Dans le cadre de ce projet, une architecture **client-serveur multicouche** a été retenue afin de garantir une séparation claire des responsabilités, une meilleure maintenabilité, une sécurité renforcée et une évolutivité facilitée.

Cette architecture permettra également d'adapter facilement l'application à de nouveaux besoins, tels que la gestion multi-établissements, la synchronisation cloud ou encore l'intégration de nouveaux services.

6.2 Architecture générale

L'application sera organisée autour de quatre couches principales :

- la couche de présentation (Frontend) ;
- la couche des services (API REST) ;
- la couche métier (Backend) ;
- la couche de persistance (Base de données).

Le fonctionnement général est le suivant :

1. L'utilisateur interagit avec l'interface Flutter.
2. Flutter envoie une requête HTTP à l'API REST.
3. L'API transmet la requête au service métier.
4. Le service métier applique les règles de gestion.
5. Les données sont lues ou enregistrées dans PostgreSQL.
6. La réponse est renvoyée au frontend qui met à jour l'interface.

Cette organisation permet de découpler totalement l'interface utilisateur de la logique métier.

6.3 Architecture logicielle

L'architecture logicielle reposera sur une architecture en couches (Layered Architecture), composée des éléments suivants.

6.3.1 Couche de présentation (Frontend)

Cette couche représente l'ensemble des interfaces graphiques utilisées par les différents utilisateurs.

Elle sera développée avec **Flutter** afin de bénéficier :

- d'une interface moderne et réactive ;
- d'une compatibilité multiplateforme (Android, Windows, Web, Linux et macOS) ;
- d'un développement à partir d'un code source unique.

Elle prendra en charge :

- l'affichage des écrans ;
- les interactions utilisateur ;
- la validation des formulaires ;
- la communication avec l'API REST ;
- la gestion locale de certaines données temporaires.

6.3.2 Couche API (REST)

Cette couche assurera la communication entre le frontend et le backend.

Ses principales responsabilités seront :

- recevoir les requêtes HTTP ;
- authentifier les utilisateurs ;
- contrôler les autorisations ;
- transmettre les demandes aux services métiers ;
- retourner les réponses au format JSON.

L'utilisation d'une API REST favorisera l'interopérabilité et facilitera l'ajout futur d'autres applications clientes.

6.3.3 Couche métier (Business Layer)

Cette couche constitue le cœur de l'application.

Elle implémente toutes les règles de gestion définies dans le cahier des charges.

Parmi ses responsabilités :

- calcul des montants des ventes ;

- mise à jour automatique des stocks ;
- validation des paiements ;
- génération des statistiques ;
- gestion des utilisateurs ;
- contrôle des permissions ;
- génération des rapports.

Cette couche garantit que toutes les opérations respectent les règles métier de l'établissement.

6.3.4 Couche de persistance

Cette couche est responsable de la gestion des données.

Elle permettra :

- l'enregistrement des informations ;
- la consultation des données ;
- les mises à jour ;
- les suppressions logiques ;
- les sauvegardes.

La base de données utilisée sera **PostgreSQL**, reconnue pour sa robustesse, sa sécurité et ses performances.

6.4 Architecture physique

L'architecture physique décrit les différents équipements qui participeront au fonctionnement du système.

Les principaux éléments sont :

- postes de caisse ;
- ordinateurs de gestion ;
- tablettes ;
- smartphones (éventuellement) ;
- serveur hébergeant l'API ;
- serveur de base de données ;
- imprimante thermique ;

- lecteur de codes-barres ;
- tiroir-caisse.

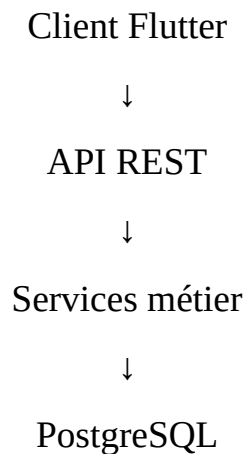
L'ensemble des équipements communiquera via un réseau local (LAN) ou une connexion Internet sécurisée selon le mode de déploiement.

6.5 Architecture réseau

Le système adoptera une architecture client-serveur.

Les clients Flutter communiqueront avec le serveur backend au moyen du protocole HTTPS.

Les échanges suivront le modèle suivant :



Cette organisation facilite :

- la sécurité ;
- la maintenance ;
- la montée en charge ;
- le déploiement à distance.

6.6 Architecture de la base de données

La base de données centralisera l'ensemble des informations manipulées par l'application.

Les principales entités seront :

- Utilisateurs
- Rôles

- Permissions
- Produits
- Catégories
- Fournisseurs
- Clients
- Stocks
- Mouvements de stock
- Achats
- Ventes
- Détails des ventes
- Paiements
- Dépenses
- Rapports
- Journaux d'activité

Les relations entre ces entités seront détaillées dans le chapitre consacré à la modélisation de la base de données.

6.7 Sécurité de l'architecture

Afin de garantir la confidentialité et l'intégrité des données, plusieurs mécanismes de sécurité seront mis en œuvre :

- authentification sécurisée des utilisateurs ;
- chiffrement des mots de passe ;
- communication sécurisée via HTTPS ;
- gestion des rôles et des permissions ;
- journalisation des opérations sensibles ;
- sauvegardes régulières de la base de données ;
- protection contre les accès non autorisés.

6.8 Évolutivité de l'architecture

L'architecture retenue permettra d'intégrer facilement de nouvelles fonctionnalités sans remettre en cause la structure générale de l'application.

Parmi les évolutions envisageables :

- gestion de plusieurs établissements ;
- synchronisation cloud ;
- application mobile dédiée aux clients ;
- prise de commandes en ligne ;
- intégration de terminaux de paiement ;
- intelligence artificielle pour l'analyse des ventes ;
- tableaux de bord décisionnels avancés ;
- connexion avec des logiciels comptables.

Cette modularité garantit la pérennité de la solution et facilite son adaptation aux besoins futurs.

L'architecture proposée répond aux exigences fonctionnelles et techniques du projet en offrant une structure claire, modulaire et évolutive. L'utilisation d'une architecture client-serveur multicouche, associée à Flutter, Django, PostgreSQL et une API REST, permettra de développer une application performante, sécurisée et facilement maintenable.

Cette architecture servira de fondement aux prochaines étapes de conception, notamment la modélisation UML, la conception de la base de données, le développement des interfaces utilisateur et l'implémentation des services backend.

CHAPITRE 7 : CHOIX TECHNOLOGIQUES

Le choix des technologies constitue une étape déterminante dans la réussite d'un projet logiciel. Il influence directement les performances, la sécurité, la maintenabilité, l'évolutivité et le coût de développement de l'application.

Dans le cadre de ce projet, les technologies retenues ont été sélectionnées en fonction de plusieurs critères, notamment leur robustesse, leur popularité, leur compatibilité, leur documentation, leur communauté de développeurs ainsi que leur capacité à répondre aux besoins fonctionnels d'une application moderne de gestion de Point de Vente (POS).

L'architecture technique repose sur une séparation claire entre le frontend, le backend et la base de données, favorisant ainsi une meilleure organisation du développement et une maintenance simplifiée.

7.2 Technologies du Frontend

Flutter

Flutter est le framework de développement multiplateforme développé par Google. Il permet de créer des interfaces utilisateur modernes et performantes à partir d'un seul code source.

Raisons du choix

- développement multiplateforme (Android, Windows, Linux, Web et macOS) ;
- excellente fluidité des interfaces graphiques ;
- richesse des composants graphiques (Widgets) ;
- productivité élevée grâce au **Hot Reload** ;
- architecture moderne et modulaire ;
- forte communauté de développeurs ;
- documentation abondante ;
- facilité d'intégration avec les API REST.

Avantages pour le projet

- une interface utilisateur professionnelle et ergonomique ;
- une réduction du temps de développement ;
- une maintenance simplifiée grâce à un code unique ;

- une évolutivité facilitée vers d'autres plateformes.

Langage Dart

Flutter utilise le langage Dart, également développé par Google.

Raisons du choix

- langage moderne orienté objet ;
- excellente intégration avec Flutter ;
- compilation rapide ;
- bonnes performances d'exécution ;
- syntaxe claire et facile à maintenir.

7.3 Technologies du Backend

Django

Le backend de l'application sera développé avec Django.

Django est un framework Java permettant de développer rapidement des applications web robustes et sécurisées.

Raisons du choix

- architecture modulaire ;
- création rapide d'API REST ;
- excellente sécurité grâce à Spring Security ;
- gestion efficace des dépendances ;
- grande stabilité en environnement professionnel ;
- vaste communauté et documentation.

Avantages

- traitement rapide des requêtes ;
- forte évolutivité ;
- maintenance facilitée ;
- intégration simple avec PostgreSQL ;
- possibilité de gérer plusieurs centaines d'utilisateurs simultanément.

Python

Python constitue le langage principal du backend.

Raisons du choix

- fiabilité ;
- robustesse ;
- programmation orientée objet ;
- excellente gestion de la mémoire ;
- très utilisé dans les applications d'entreprise.

7.4 Technologies de la base de données

PostgreSQL

PostgreSQL est un système de gestion de base de données relationnelle open source reconnu pour sa stabilité et ses performances.

Raisons du choix

- excellente fiabilité ;
- conformité aux standards SQL ;
- haute sécurité ;
- bonnes performances sur de grands volumes de données ;
- gestion avancée des transactions ;
- sauvegardes efficaces.

Avantages

- intégrité des données ;
- rapidité des requêtes ;
- évolutivité ;
- compatibilité avec Django.

7.5 Communication entre les couches

API REST

La communication entre le frontend et le backend sera assurée par une API REST.

Raisons du choix

- simplicité d'utilisation ;
- indépendance entre les applications clientes et le serveur ;
- compatibilité avec Flutter ;
- échanges de données au format JSON ;
- facilité d'intégration avec d'autres systèmes.

Format JSON

Toutes les données échangées entre le client et le serveur utiliseront le format JSON.

Avantages

- format léger ;
- facilement lisible ;
- largement adopté ;
- compatible avec la majorité des langages de programmation.

7.6 Sécurité

JWT (JSON Web Token)

Le système utilisera JWT pour gérer l'authentification des utilisateurs.

Avantages

- authentification sécurisée ;
- gestion des sessions sans état (Stateless) ;
- performances élevées ;
- intégration simple avec Spring Security.

Spring Security

Spring Security assurera :

- l'authentification des utilisateurs ;
- la gestion des rôles ;
- le contrôle des permissions ;
- la protection des routes sensibles.

7.7 Outils de développement

Visual Studio Code

Utilisé principalement pour le développement Flutter.

Avantages

- léger ;
- rapide ;
- nombreuses extensions ;
- excellente prise en charge de Flutter et Dart.

Git

Git sera utilisé pour le contrôle de version.

Avantages

- suivi des modifications ;
- collaboration entre développeurs ;
- restauration des versions précédentes ;
- gestion des branches.

GitHub

GitHub permettra :

- l'hébergement du code source ;
- la collaboration ;
- la sauvegarde du projet ;
- la gestion des versions.

Postman

Postman sera utilisé pour :

- tester les API REST ;
- vérifier les réponses du serveur ;
- faciliter le développement backend.

Figma

Figma servira à concevoir les maquettes graphiques de l'application.

Avantages

- conception collaborative ;
- prototypage interactif ;
- facilité de modification ;
- exportation des composants.

7.8 Technologies complémentaires

Selon les besoins futurs, les technologies suivantes pourront être intégrées :

- Docker pour le déploiement et la conteneurisation ;
- Nginx comme serveur web et reverse proxy ;
- Redis pour la mise en cache ;
- Firebase Cloud Messaging pour les notifications push ;
- Cloud Storage pour les sauvegardes distantes ;
- Grafana et Prometheus pour la supervision des performances.

7.9 Compatibilité du système

L'application sera compatible avec :

- Android ;
- Windows ;
- Linux ;
- macOS ;
- Web (version future).

Elle pourra fonctionner sur :

- ordinateurs de bureau ;
- ordinateurs portables ;
- tablettes ;
- terminaux tactiles utilisés en caisse.

7.10 Synthèse des choix technologiques

Composant	Technologie retenue	Justification principale
Frontend	Flutter	Développement multiplateforme, interface moderne et performante
Langage Frontend	Dart	Intégration native avec Flutter
Backend	Django	Robustesse, modularité et sécurité
Langage Backend	Python	Fiabilité et performances
Base de données	PostgreSQL	Stabilité, sécurité et conformité SQL
Communication	API REST	Interopérabilité et simplicité
Format d'échange	JSON	Léger et universel
Authentification	JWT	Sécurité et gestion des sessions
Sécurité	Spring Security	Gestion des accès et des permissions
Contrôle de version	Git	Gestion des versions
Hébergement du code	GitHub	Collaboration et sauvegarde
Tests API	Postman	Validation des services REST
Conception UI/UX	Figma	Maquettage et prototypage

Les technologies retenues répondent aux exigences fonctionnelles et techniques du projet. Elles offrent une architecture moderne, modulaire et évolutive, capable de supporter les opérations d'un système de Point de Vente destiné aux débits de boissons.

Le choix de Flutter pour le frontend, de Django pour le backend, de PostgreSQL pour la gestion des données et des technologies complémentaires telles que JWT, Git et Postman garantit le développement d'une application performante, sécurisée et facilement maintenable. Cette base technologique permettra également d'intégrer de nouvelles fonctionnalités dans les versions futures sans remettre en cause l'architecture générale du système.

CHAPITRE 8 : MODÉLISATION UML

La modélisation UML (Unified Modeling Language) est une étape essentielle dans la conception d'un système informatique. Elle permet de représenter de manière graphique les différentes composantes du système, leurs interactions ainsi que les processus métiers qui seront implémentés lors du développement.

Dans le cadre de ce projet, la modélisation UML a pour objectif de traduire les besoins fonctionnels identifiés dans les chapitres précédents en modèles compréhensibles par les développeurs et les parties prenantes. Elle facilite également la communication entre les membres de l'équipe de développement, réduit les risques d'erreurs de conception et constitue une référence tout au long du cycle de vie du projet.

Les principaux diagrammes UML retenus pour ce projet sont :

- le diagramme de cas d'utilisation ;
- le diagramme de classes ;
- les diagrammes de séquence ;
- les diagrammes d'activités ;
- le diagramme d'états ;
- le diagramme de composants ;

Ces différents diagrammes permettront de décrire aussi bien la structure que le comportement de l'application de gestion de Point de Vente (POS).

8.2 Diagramme de cas d'utilisation général

8.2.1 Objectif

Le diagramme de cas d'utilisation présente une vue globale des fonctionnalités offertes par le système ainsi que les interactions entre les différents acteurs et l'application.

Il constitue le point de départ de la conception UML puisqu'il permet d'identifier les services attendus par chaque type d'utilisateur.

8.2.2 Acteurs du système

Le système comporte les acteurs principaux suivants.

Administrateur

L'administrateur dispose de tous les privilèges. Il assure la configuration du système, la gestion des utilisateurs, des rôles, des paramètres et supervise l'ensemble des opérations.

Gérant

Le gérant supervise les activités commerciales de l'établissement. Il consulte les rapports, valide certaines opérations sensibles et assure le suivi des performances.

Caissier

Le caissier est responsable de l'enregistrement des ventes, des paiements, de l'ouverture et de la fermeture de caisse ainsi que de l'impression des reçus.

Magasinier (Logisticien)

Le magasinier gère les approvisionnements, les mouvements de stock, les inventaires et le suivi des produits.

Serveur

Le serveur peut enregistrer les commandes des clients et les transmettre au caissier selon l'organisation de l'établissement.

Comptable

Le comptable consulte les rapports financiers, les dépenses, les achats et les statistiques nécessaires au suivi comptable.

8.2.3 Cas d'utilisation du système

Le système devra offrir les fonctionnalités suivantes.

Authentification

- Se connecter
- Se déconnecter
- Modifier son mot de passe
- Réinitialiser un mot de passe

Gestion des utilisateurs

- Ajouter un utilisateur
- Modifier un utilisateur

- Désactiver un utilisateur
- Supprimer un utilisateur
- Attribuer un rôle
- Gérer les permissions

Gestion des catégories

- Ajouter une catégorie
- Modifier une catégorie
- Supprimer une catégorie
- Consulter les catégories

Gestion des produits

- Ajouter un produit
- Modifier un produit
- Supprimer un produit
- Rechercher un produit
- Consulter un produit

Gestion des fournisseurs

- Ajouter un fournisseur
- Modifier un fournisseur
- Supprimer un fournisseur
- Consulter un fournisseur

Gestion des achats

- Enregistrer un achat
- Réceptionner une livraison
- Annuler un achat
- Consulter l'historique des achats

Gestion des stocks

- Enregistrer une entrée
- Enregistrer une sortie

- Réaliser un inventaire
- Consulter les mouvements
- Corriger un stock

Gestion des ventes (POS)

- Ouvrir une caisse
- Fermer une caisse
- Créer une vente
- Ajouter un article
- Modifier une quantité
- Supprimer un article
- Annuler une vente
- Valider une vente
- Imprimer un reçu

Gestion des paiements

- Encaisser un paiement
- Choisir un mode de paiement
- Calculer la monnaie
- Imprimer un reçu de paiement

Gestion des clients

- Ajouter un client
- Modifier un client
- Rechercher un client
- Consulter l'historique

Gestion des dépenses

- Ajouter une dépense
- Modifier une dépense
- Supprimer une dépense
- Consulter les dépenses

Rapports

- Consulter les statistiques
- Générer un rapport
- Exporter un rapport
- Imprimer un rapport

Paramètres

- Configurer l'entreprise
- Configurer les taxes
- Configurer les imprimantes
- Effectuer une sauvegarde
- Restaurer une sauvegarde

8.2.4 Description textuelle des interactions

Chaque acteur interagit avec le système selon les responsabilités qui lui sont attribuées.

L'administrateur dispose d'un accès complet à l'ensemble des modules. Il est chargé de la configuration générale du système, de la gestion des utilisateurs, des rôles, des paramètres et de la supervision des opérations.

Le gérant intervient principalement dans la consultation des tableaux de bord, la gestion des produits, des fournisseurs, des stocks ainsi que dans la validation des opérations nécessitant une autorisation particulière.

Le caissier utilise essentiellement le module de Point de Vente afin d'enregistrer les ventes, d'encaisser les paiements, d'imprimer les reçus et de gérer les ouvertures et fermetures de caisse.

Le magasinier gère les approvisionnements, les mouvements de stock et les inventaires.

Le serveur enregistre les commandes des clients et les transmet au caissier pour le règlement.

Le comptable consulte les rapports financiers, les dépenses, les ventes et les statistiques nécessaires au suivi comptable.

8.2.5 Représentation du diagramme

Le diagramme de cas d'utilisation général fera apparaître :

- les six acteurs principaux ;
- la frontière représentant le système « Application POS pour Débits de Boissons » ;
- l'ensemble des cas d'utilisation identifiés ;
- les relations d'association entre les acteurs et les cas d'utilisation ;
- les relations <> et <> lorsque cela sera nécessaire.

Ce diagramme constituera une vue synthétique des interactions entre les utilisateurs et le système et servira de base à la réalisation des diagrammes de cas d'utilisation détaillés par module.

Le diagramme de cas d'utilisation général permet d'identifier les fonctionnalités majeures de l'application et les interactions entre les différents acteurs du système. Il constitue la première étape de la modélisation UML et servira de référence pour la conception des diagrammes détaillés, du diagramme de classes, des diagrammes de séquence ainsi que de la base de données.

CHAPITRE 9 : CONCEPTION DE LA BASE DE DONNÉES

La base de données constitue le cœur du système d'information. Elle assure le stockage, l'organisation et la gestion de l'ensemble des données manipulées par l'application de gestion de Point de Vente (POS). Une conception rigoureuse est indispensable afin de garantir la cohérence, la sécurité et la disponibilité des informations.

Dans le cadre de ce projet, une base de données relationnelle sera utilisée. Le système de gestion de base de données retenu est PostgreSQL, reconnu pour sa fiabilité, sa robustesse et ses performances dans les applications professionnelles.

La conception de la base de données suivra une approche progressive comprenant le dictionnaire des données, le Modèle Conceptuel de Données (MCD), le Modèle Logique de Données (MLD) et le schéma relationnel.

9.2 Objectifs de la base de données

La base de données devra permettre de :

- centraliser toutes les informations de l'application ;
- assurer l'intégrité et la cohérence des données ;
- faciliter les recherches et les traitements ;
- garantir la traçabilité des opérations ;
- sécuriser les informations sensibles ;
- permettre l'évolution future du système.

9.3 Dictionnaire des données

Le dictionnaire des données décrit les principales entités manipulées par le système ainsi que leur rôle.

Entité	Description
Utilisateur	Personne autorisée à utiliser l'application
Rôle	Ensemble des permissions attribuées à un utilisateur
Permission	Droit d'accès à une fonctionnalité
Catégorie	Classe de produits (bières, vins, jus, etc.)
Produit	Article vendu par l'établissement
Fournisseur	Entreprise ou personne fournissant les produits
Achat	Opération d'approvisionnement
Détail Achat	Produits composant un achat
Stock	Quantité disponible d'un produit
Mouvement Stock	Historique des entrées et sorties de stock
Client	Client enregistré dans le système
Vente	Transaction commerciale
Détail Vente	Produits composant une vente
Paieement	Informations relatives au règlement d'une vente
Dépense	Dépenses de fonctionnement
Caisse	Informations sur les ouvertures et fermetures de caisse
Journal Activité	Historique des actions réalisées dans le système

9.4 Modèle Conceptuel de Données (MCD)

Le MCD représente les principales entités du système et les relations qui existent entre elles, indépendamment de toute technologie.

Les principales relations sont les suivantes :

- Un rôle peut être attribué à plusieurs utilisateurs.
- Un utilisateur possède un seul rôle.
- Une catégorie regroupe plusieurs produits.
- Un produit appartient à une seule catégorie.
- Un fournisseur peut fournir plusieurs produits.
- Un achat est effectué auprès d'un fournisseur.
- Un achat contient plusieurs produits.

- Une vente est réalisée par un utilisateur.
- Une vente peut concerner un client.
- Une vente est composée de plusieurs lignes de vente.
- Chaque ligne de vente correspond à un produit.
- Une vente peut être réglée par un ou plusieurs paiements.
- Chaque mouvement de stock est associé à un produit.
- Une caisse est ouverte et fermée par un caissier.

Le MCD sera représenté graphiquement à l'aide d'un diagramme entité-association.

9.5 Modèle Logique de Données (MLD)

Le MLD traduit le MCD en tables relationnelles.

Les principales tables prévues sont :

- utilisateurs
- roles
- permissions
- role_permissions
- categories
- produits
- fournisseurs
- achats
- detail_achats
- stocks
- mouvements_stock
- clients
- ventes
- detail_ventes
- paiements
- dépenses
- caisses

- journaux_activite

Chaque table comportera une clé primaire permettant d'identifier de manière unique chaque enregistrement, ainsi que des clés étrangères assurant les relations entre les différentes tables.

9.6 Description des principales tables

Table Utilisateurs

Cette table stocke les informations relatives aux utilisateurs de l'application.

Principaux attributs :

- id_utilisateur
- nom
- prénom
- téléphone
- email
- nom_utilisateur
- mot_de_passe
- photo
- statut
- date_creation
- role_id

Table Rôles

Elle définit les différents profils d'utilisation.

Attributs :

- id_role
- nom_role
- description

Table Produits

Elle contient les informations sur les boissons et autres articles.

Attributs :

- id_produit
- code
- code_barres
- nom
- description
- prix_achat
- prix_vente
- quantité
- stock_minimum
- image
- categorie_id
- fournisseur_id

Table Catégories

Elle permet de classer les produits.

Attributs :

- id_categorie
- nom
- description

Table Fournisseurs

Elle enregistre les partenaires commerciaux.

Attributs :

- id_fournisseur
- raison_sociale
- téléphone
- adresse
- email

Table Achats

Elle enregistre les opérations d'approvisionnement.

Attributs :

- id_achat
- date_achat
- montant_total
- fournisseur_id
- utilisateur_id

Table Détail Achats

Elle détaille les produits composant un achat.

Attributs :

- id_detail
- achat_id
- produit_id
- quantité
- prix_unitaire

Table Ventes

Elle enregistre les transactions commerciales.

Attributs :

- id_vente
- référence
- date_vente
- montant_total
- remise
- statut
- utilisateur_id
- client_id
- caisse_id

Table Détail Ventes

Elle contient les lignes de chaque vente.

Attributs :

- id_detail
- vente_id
- produit_id
- quantité
- prix_unitaire
- sous_total

Table Paiements

Elle enregistre les règlements.

Attributs :

- id_paiement
- vente_id
- mode_paiement
- montant
- référence_transaction
- date_paiement

Table Stocks

Elle contient les quantités disponibles.

Attributs :

- id_stock
- produit_id
- quantité_disponible
- seuil_alerte

Table Mouvements Stock

Elle historise tous les mouvements.

Attributs :

- id_mouvement
- produit_id

- type_mouvement
- quantité
- date_mouvement
- utilisateur_id

Table Dépenses

Elle enregistre les dépenses de fonctionnement.

Attributs :

- id_depense
- libellé
- catégorie
- montant
- date_depense
- utilisateur_id

Table Caisses

Elle assure le suivi des ouvertures et fermetures de caisse.

Attributs :

- id_caisse
- date_ouverture
- date_fermeture
- montant_initial
- montant_final
- utilisateur_id

Table Journal Activité

Elle conserve l'historique des actions réalisées.

Attributs :

- id_journal
- utilisateur_id
- action

- description
- date_action
- adresse_ip

9.7 Contraintes d'intégrité

Afin de garantir la qualité des données, les règles suivantes seront appliquées :

- toutes les clés primaires seront uniques ;
- les clés étrangères assureront la cohérence des relations ;
- les champs obligatoires ne pourront pas être vides ;
- les montants devront être positifs ;
- les quantités de stock ne pourront pas être négatives ;
- la suppression de données sensibles sera contrôlée afin de préserver la traçabilité.

9.8 Sécurité des données

La base de données intégrera plusieurs mécanismes de sécurité :

- chiffrement des mots de passe ;
- contrôle des accès selon les rôles ;
- sauvegardes automatiques ;
- journalisation des opérations importantes ;
- protection contre les suppressions accidentelles.

La conception de la base de données proposée répond aux besoins fonctionnels identifiés dans les chapitres précédents. Elle offre une structure relationnelle cohérente, normalisée et évolutive, capable de gérer efficacement les utilisateurs, les produits, les ventes, les stocks, les paiements et les autres données de l'application.

Cette conception servira de base à la réalisation du diagramme de classes UML, du schéma relationnel détaillé et de l'implémentation de la base de données PostgreSQL lors de la phase de développement.

CHAPITRE 10 : CONCEPTION DES INTERFACES UTILISATEUR (UI/UX)

L'interface utilisateur constitue le point de contact entre les utilisateurs et l'application. Une interface bien conçue améliore l'expérience utilisateur, réduit le temps d'apprentissage et limite les erreurs lors de l'utilisation du système.

Dans le cadre de ce projet, une attention particulière est accordée à l'ergonomie, à la simplicité de navigation, à la cohérence visuelle et à la rapidité d'exécution des tâches. Les interfaces seront développées avec Flutter afin d'offrir une expérience fluide et homogène sur différentes plateformes.

Les maquettes présentées dans ce chapitre illustrent les principaux écrans de l'application et décrivent leurs fonctionnalités.

10.2 Principes de conception UI/UX

La conception des interfaces repose sur plusieurs principes fondamentaux :

Simplicité

Les écrans devront être clairs, épurés et faciles à comprendre, même pour un utilisateur débutant.

Ergonomie

Les informations importantes devront être immédiatement visibles et les actions les plus fréquentes accessibles en un minimum de clics.

Cohérence

Toutes les interfaces utiliseront une charte graphique uniforme (couleurs, typographies, icônes, boutons et formulaires).

Réactivité

L'application devra s'adapter aux différentes tailles d'écran (ordinateur, tablette et terminal tactile).

Accessibilité

Les textes seront lisibles, les boutons suffisamment grands et les contrastes visuels adaptés à une utilisation prolongée.

10.3 Charte graphique

Afin d'assurer une identité visuelle professionnelle, l'application adoptera une charte graphique moderne.

Palette de couleurs

- Couleur principale : Bleu (navigation et en-têtes)
- Couleur secondaire : Orange (actions principales)
- Couleur de validation : Vert
- Couleur d'alerte : Jaune
- Couleur d'erreur : Rouge
- Fond principal : gris très clair

Typographie

- Police moderne et lisible.
- Titres en gras.
- Taille adaptée aux écrans tactiles.

Icônes

Des icônes explicites seront utilisées pour représenter les différentes fonctionnalités (vente, stock, produits, utilisateurs, rapports, paramètres, etc.).

10.4 Écran de connexion

Objectif

Permettre aux utilisateurs autorisés d'accéder au système.

Composants

- Logo de l'application
- Nom de l'établissement
- Champ "Nom d'utilisateur"
- Champ "Mot de passe"
- Bouton **Se connecter**
- Option **Afficher/Masquer le mot de passe**
- Lien **Mot de passe oublié** (si activé)

Fonctionnalités

- Validation des informations saisies

- Authentification sécurisée
- Gestion des erreurs de connexion
- Redirection vers le tableau de bord selon le rôle

10.5 Tableau de bord

Objectif

Présenter un résumé de l'activité de l'établissement.

Informations affichées

- Chiffre d'affaires du jour
- Nombre de ventes
- Produits en rupture
- Alertes de stock
- Dépenses du jour
- Produits les plus vendus
- Graphique des ventes
- Graphique des bénéfices

Accès rapides

- Nouvelle vente
- Ajouter un produit
- Consulter le stock
- Générer un rapport

10.6 Interface de vente (POS)

Objectif

Permettre au caissier d'effectuer rapidement les ventes.

Composants

- Barre de recherche
- Lecture du code-barres
- Liste des catégories

- Catalogue des produits
- Panier
- Calcul automatique du total
- Bouton **Paielement**
- Bouton **Annuler**
- Bouton **Imprimer le reçu**

Fonctionnalités

- Ajout rapide des produits
- Modification des quantités
- Suppression d'un article
- Application d'une remise
- Calcul automatique du montant
- Impression du ticket

10.7 Gestion des produits

Objectif

Administrer les produits commercialisés.

Éléments de l'écran

- Tableau des produits
- Bouton Ajouter
- Bouton Modifier
- Bouton Supprimer
- Champ de recherche
- Filtre par catégorie

Informations affichées

- Nom
- Catégorie
- Prix de vente

- Stock
- Statut

10.8 Gestion des catégories

Cette interface permettra :

- d'ajouter une catégorie ;
- de modifier une catégorie ;
- de supprimer une catégorie ;
- d'afficher la liste des catégories.

10.9 Gestion des stocks

L'écran de gestion des stocks affichera :

- les produits disponibles ;
- les quantités restantes ;
- le seuil minimal ;
- les alertes de rupture ;
- l'historique des mouvements.

Des actions permettront de :

- enregistrer une entrée ;
- enregistrer une sortie ;
- réaliser un inventaire.

10.10 Gestion des fournisseurs

L'interface permettra :

- d'ajouter un fournisseur ;
- de modifier ses informations ;
- de consulter son historique ;
- d'enregistrer une livraison.

10.11 Gestion des clients

L'écran présentera :

- les informations du client ;
- son historique d'achats ;
- le montant total des achats ;
- les statistiques de fidélité (version future).

10.12 Gestion des achats

Cette interface permettra :

- d'enregistrer un approvisionnement ;
- de sélectionner un fournisseur ;
- d'ajouter les produits livrés ;
- de mettre automatiquement le stock à jour.

10.13 Gestion des paiements

L'utilisateur pourra :

- choisir le mode de paiement ;
- saisir le montant reçu ;
- calculer automatiquement la monnaie ;
- imprimer le reçu.

Modes de paiement disponibles :

- Espèces
- Mobile Money
- Carte bancaire
- Paiement mixte

10.14 Gestion des dépenses

L'interface offrira les fonctionnalités suivantes :

- ajout d'une dépense ;
- modification ;
- suppression ;
- consultation de l'historique.

10.15 Rapports et statistiques

Les responsables pourront consulter :

- les ventes journalières ;
- les ventes mensuelles ;
- les bénéfices ;
- les dépenses ;
- les achats ;
- les produits les plus vendus ;
- les performances des caissiers.

Les rapports pourront être :

- imprimés ;
- exportés en PDF ;
- exportés en Excel.

10.16 Paramètres

L'écran des paramètres permettra de :

- modifier les informations de l'entreprise ;
- gérer les utilisateurs ;
- configurer les taxes ;
- configurer les imprimantes ;
- gérer les sauvegardes ;
- personnaliser l'application.

10.17 Responsive Design

Les interfaces seront adaptées à plusieurs types d'équipements :

- ordinateur de bureau ;
- ordinateur portable ;
- tablette ;
- écran tactile de caisse.

Les composants graphiques s'ajusteront automatiquement à la résolution de l'écran afin de garantir une expérience utilisateur optimale.

10.18 Parcours utilisateur

Le parcours principal de l'utilisateur sera le suivant :

1. Connexion au système.
2. Accès au tableau de bord.
3. Sélection du module souhaité.
4. Réalisation des opérations (vente, stock, achat, etc.).
5. Validation et enregistrement.
6. Consultation des rapports.
7. Déconnexion.

La conception des interfaces utilisateur proposée répond aux exigences d'ergonomie, de simplicité et d'efficacité attendues d'une application moderne de Point de Vente. Les différents écrans permettront aux utilisateurs d'effectuer leurs tâches quotidiennes de manière rapide et intuitive, tout en assurant une navigation fluide entre les différents modules du système.

Les maquettes décrites dans ce chapitre serviront de référence lors du développement des interfaces avec Flutter et pourront être enrichies au fur et à mesure des phases de réalisation et de validation du projet.

CHAPITRE 11 : PLANIFICATION DU PROJET

11.1 Introduction

La planification constitue une étape essentielle dans la conduite d'un projet informatique. Elle permet d'organiser les différentes activités, de définir les ressources nécessaires, d'estimer les délais de réalisation et d'assurer un suivi efficace de l'avancement du projet.

Dans le cadre du développement de l'application de gestion de Point de Vente (POS) pour les débits de boissons, une planification rigoureuse est indispensable afin de garantir le respect des objectifs, des délais et de la qualité attendue.

Ce chapitre présente la méthodologie de développement adoptée, les ressources mobilisées, le découpage du projet en phases, le calendrier prévisionnel ainsi que les principaux risques identifiés.

11.2 Méthodologie de développement

Le projet sera conduit selon une approche **Agile**, plus précisément inspirée de la méthode **Scrum**.

Cette approche a été retenue pour les raisons suivantes :

- développement progressif par itérations ;
- intégration continue des améliorations ;
- adaptation rapide aux changements ;
- validation régulière des fonctionnalités ;
- meilleure collaboration entre les parties prenantes.

Chaque itération permettra de développer, tester et valider un ensemble de fonctionnalités avant de passer à la suivante.

11.3 Découpage du projet

Le projet est organisé en plusieurs phases.

Phase 1 : Analyse des besoins

Objectifs :

- recueil des besoins ;
- étude de l'existant ;
- rédaction du cahier des charges ;

- validation des exigences.

Livrables :

- cahier des charges ;
- spécifications fonctionnelles.

Phase 2 : Conception

Objectifs :

- modélisation UML ;
- conception de la base de données ;
- conception de l'architecture logicielle ;
- réalisation des maquettes UI/UX.

Livrables :

- diagrammes UML ;
- schéma relationnel ;
- maquettes des interfaces.

Phase 3 : Développement du backend

Objectifs :

- mise en place de l'architecture Spring Boot ;
- développement des API REST ;
- implémentation de la logique métier ;
- sécurisation des accès.

Livrables :

- API REST fonctionnelle ;
- documentation technique.

Phase 4 : Développement du frontend

Objectifs :

- création des interfaces Flutter ;
- intégration avec les API ;
- gestion de la navigation ;

- implémentation des fonctionnalités utilisateur.

Livrables :

- application Flutter fonctionnelle.

Phase 5 : Tests

Objectifs :

- tests unitaires ;
- tests d'intégration ;
- tests fonctionnels ;
- correction des anomalies.

Livrables :

- rapports de tests ;
- version stabilisée.

Phase 6 : Déploiement

Objectifs :

- préparation de l'environnement ;
- installation de l'application ;
- configuration de la base de données ;
- formation des utilisateurs.

Livrables :

- application opérationnelle ;
- documentation utilisateur.

Phase 7 : Maintenance

Objectifs :

- correction des anomalies ;
- optimisation des performances ;
- évolution fonctionnelle ;
- assistance aux utilisateurs.

Livrables :

- mises à jour ;
- nouvelles versions.

11.4 Ressources nécessaires

Ressources humaines

Le projet mobilise les profils suivants :

- Chef de projet ;
- Analyste fonctionnel ;
- Architecte logiciel ;
- Développeur Backend ;
- Développeur Flutter ;
- Testeur ;
- Administrateur de base de données ;
- Utilisateurs pilotes.

Dans le cadre d'un projet académique, plusieurs de ces rôles peuvent être assurés par une même personne.

Ressources matérielles

Les principaux équipements nécessaires sont :

- ordinateur de développement ;
- connexion Internet ;
- serveur de développement ;
- smartphone Android pour les tests ;
- imprimante thermique (tests d'impression) ;
- lecteur de codes-barres ;
- terminal de paiement (si disponible).

Ressources logicielles

Les outils utilisés sont :

- Flutter ;
- Dart ;

- Python ;
- Django ;
- PostgreSQL ;
- Git ;
- GitHub ;
- Visual Studio Code ;
- Postman ;
- Figma.

11.5 Planning prévisionnel

Le projet peut être organisé selon le planning suivant :

Phase	Activités principales	Durée estimée
Analyse des besoins	Cahier des charges et spécifications	3 jours
Conception	UML, base de données, architecture, maquettes	1 semaine
Développement Backend	API REST et logique métier	1 jour
Développement Frontend	Interfaces Flutter et intégration	1 jour
Tests	Validation et corrections	2 jours
Déploiement	Installation et configuration	Pas besoin
Documentation finale	Manuel utilisateur et rapport	1 jour
Durée totale estimée : 2 semaines .		

11.6 Gestion des risques

Plusieurs risques peuvent affecter le bon déroulement du projet.

Risque	Impact	Mesures préventives
Retard de développement	Élevé	Planification détaillée et suivi régulier
Évolution des besoins	Moyen	Méthodologie Agile et validation continue
Perte de données	Élevé	Sauvegardes automatiques et gestion de versions
Défaillance matérielle	Moye	Sauvegarde du code sur GitHub

Risque	Impact	Mesures préventives
	n	
Difficultés techniques	Moyenne	Documentation et assistance communautaire
Bugs critiques	Élevé	Tests unitaires et tests d'intégration

11.7 Critères de réussite

Le projet sera considéré comme réussi si :

- toutes les fonctionnalités prévues sont implémentées ;
- les performances répondent aux besoins des utilisateurs ;
- les données sont sécurisées ;
- les tests sont concluants ;
- les interfaces sont ergonomiques ;
- la documentation est complète ;
- l'application est stable et prête à être utilisée.

11.8 Suivi et évaluation

Le suivi du projet reposera sur :

- des réunions de suivi régulières ;
- un tableau de bord d'avancement ;
- des démonstrations à la fin de chaque itération ;
- une gestion des versions avec Git ;
- un suivi des anomalies et des corrections.

Cette démarche permettra de contrôler l'avancement du projet et de garantir la qualité des livrables.

11.9 Répartition des tâches

Répartition générale du projet

Équipe GSI (7 membres)

Responsabilité : Architecture, Backend, Base de données, API REST, Sécurité et Déploiement.

Équipe GSA (2 membres)

Responsabilité : UI/UX, Frontend Flutter, Intégration des API et Expérience utilisateur.

ÉQUIPE GSI (7 membres)

AKOA MENGUE Verone — Chef technique & Architecte logiciel

Responsabilités

- Coordination technique
- Architecture du projet Django
- Organisation du dépôt Git
- Revue de code
- Intégration backend
- Documentation technique

Livrables

- Architecture backend
- Documentation technique
- Guide de développement

SEGNING WOUATI Gildas — Analyste & Modélisation

Responsabilités

- Diagrammes UML
- Cas d'utilisation
- Diagrammes de classes
- Diagrammes de séquence
- Diagrammes d'activités
- Diagrammes de déploiement
- Documentation UML

Livrables

- Dossier UML complet

NJOFANG OTTOU Michel — Base de données PostgreSQL

Responsabilités

- MCD
- MLD
- Schéma relationnel
- Création des tables
- Relations
- Contraintes
- Scripts SQL
- Optimisation

Livrables

- Base de données PostgreSQL
- Scripts SQL

NJOFANG OTTOU Michel & EBODE Yvan — Authentification & Sécurité

Responsabilités

- Authentification
- JWT
- Permissions
- Rôles
- Gestion des utilisateurs
- Sécurité Django

Modules

- Authentication
- Users
- Roles

ELOM Lydie Patricia — Gestion commerciale

Responsabilités

- Produits
- Catégories
- Fournisseurs
- Clients

Modules

- Products
- Categories
- Suppliers
- Customers

MASSE MASSE Paul & SEGNING WOUATI Gildas — Gestion opérationnelle

Responsabilités

- Ventes
- Achats
- Stocks
- Paiements
- Dépenses

Modules

- Sales
- Purchases
- Inventory
- Payments
- Expenses

BALLA MESSADOM Laetitia — Rapports, Tests

Responsabilités

- Tableau de bord
- Rapports
- Statistiques
- Tests backend
- Documentation API
- Déploiement

Modules

- Reports
- Dashboard
- Notifications

ÉQUIPE GSA (2 membres)

BOUBA — UI/UX Designer & Développeur Flutter

Responsabilités

- Charte graphique
- Design System
- Maquettes (Figma)
- Splash Screen
- Authentification
- Tableau de bord
- Navigation
- Widgets réutilisables
- Responsive Design

Livrables

- Design System
- Maquettes
- Navigation Flutter
- Écrans principaux

OSSOM — Développeur Flutter & Intégration

Responsabilités

- Interface POS
- Produits
- Catégories
- Fournisseurs
- Clients
- Stocks
- Achats
- Rapports
- Paramètres
- Intégration avec les API
- Tests du frontend

Livrables

- Modules Flutter
- Intégration API

- Documentation utilisateur

Collaboration GSI ↔ GSA

Les deux équipes devront travailler ensemble sur :

- Définition des contrats d'API (endpoints, formats JSON).
- Tests d'intégration frontend/backend.
- Validation des fonctionnalités.
- Corrections des anomalies.
- Déploiement final.

La planification proposée permet de structurer le développement de l'application POS en plusieurs phases cohérentes, depuis l'analyse des besoins jusqu'au déploiement et à la maintenance. L'adoption d'une méthodologie Agile, associée à une bonne gestion des ressources et des risques, favorisera la réalisation d'une application fiable, évolutive et répondant aux attentes des futurs utilisateurs.

Chapitre 12 : LIVRABLES DU PROJET

Les livrables représentent l'ensemble des documents, modèles et logiciels qui seront produits au cours du développement de l'application de gestion de Point de Vente (POS) pour les débits de boissons. Ils constituent les résultats concrets attendus à chaque étape du projet et permettront de valider son avancement ainsi que sa conformité aux exigences définies dans le présent cahier des charges.

Les principaux livrables sont les suivants :

1. Documents de gestion du projet

- Cahier des charges fonctionnel et technique ;
- Planning de réalisation du projet ;
- Documentation des exigences fonctionnelles et non fonctionnelles ;
- Documentation de l'architecture logicielle.

2. Documents de conception

- Diagramme de cas d'utilisation ;
- Diagrammes de classes ;
- Diagrammes de séquence ;
- Diagrammes d'activités ;
- Diagrammes d'états ;
- Diagramme de composants ;
- Diagramme de déploiement ;
- Modèle Conceptuel de Données (MCD) ;
- Modèle Logique de Données (MLD) ;
- Schéma relationnel de la base de données.

3. Maquettes et interfaces

- Maquette de l'écran de connexion ;
- Maquette du tableau de bord ;
- Maquette de l'interface de vente (POS) ;
- Maquette de gestion des produits ;
- Maquette de gestion des catégories ;

- Maquette de gestion des stocks ;
- Maquette de gestion des fournisseurs ;
- Maquette de gestion des clients ;
- Maquette de gestion des utilisateurs ;
- Maquette des rapports et statistiques ;
- Maquette des paramètres de l'application.

4. Base de données

- Script de création de la base de données PostgreSQL ;
- Scripts de création des tables ;
- Contraintes d'intégrité ;
- Jeux de données de test ;
- Procédures de sauvegarde et de restauration.

5. Développement Backend

- Projet Django complet ;
- API REST documentée ;
- Services métiers ;
- Gestion de l'authentification et des autorisations ;
- Documentation des API.

6. Développement Frontend

- Application Flutter complète ;
- Interfaces utilisateur fonctionnelles ;
- Navigation entre les écrans ;
- Intégration avec les API REST ;
- Gestion des états de l'application.

7. Fonctionnalités réalisées

L'application devra permettre :

- l'authentification sécurisée des utilisateurs ;

- la gestion des rôles et des permissions ;
- la gestion des produits ;
- la gestion des catégories ;
- la gestion des fournisseurs ;
- la gestion des achats ;
- la gestion des stocks ;
- la gestion des ventes ;
- la gestion des paiements ;
- la gestion des clients ;
- la gestion des dépenses ;
- la gestion des caisses ;
- la génération de rapports ;
- l'impression des reçus ;
- la sauvegarde et la restauration des données.

8. Documentation

- Manuel utilisateur ;
- Manuel administrateur ;
- Documentation technique ;
- Guide d'installation ;
- Guide de déploiement ;
- Guide de maintenance.

9. Tests

- Plan de tests ;
- Rapports de tests unitaires ;
- Rapports de tests d'intégration ;
- Rapports de tests fonctionnels ;
- Rapport de validation finale.

CONCLUSION GÉNÉRALE

Ce projet de conception d'une application de gestion de Point de Vente (POS) destinée aux débits de boissons s'inscrit dans une démarche de modernisation et d'optimisation des systèmes de gestion commerciale. L'étude réalisée tout au long de ce cahier des charges a permis de mettre en évidence les limites des méthodes traditionnelles de gestion ainsi que la nécessité d'adopter une solution numérique intégrée, fiable et évolutive.

L'analyse des besoins a conduit à la définition de fonctionnalités essentielles couvrant l'ensemble des activités d'un établissement, notamment la gestion des ventes, des stocks, des achats, des produits, des fournisseurs, des clients, des paiements et des dépenses. À cela s'ajoute la production de rapports et de tableaux de bord permettant une meilleure prise de décision.

La conception du système a été structurée autour d'une architecture moderne basée sur une séparation claire entre le frontend, le backend et la base de données. Le choix de technologies telles que Flutter, Django et PostgreSQL garantit à la fois performance, sécurité, maintenabilité et évolutivité.

Les différentes étapes de modélisation UML, de conception de la base de données, de définition des interfaces utilisateur et de planification du projet ont permis de structurer rigoureusement le système avant sa réalisation. Cette approche assure la cohérence globale de la solution et facilite son développement.

En perspective, l'application pourra évoluer vers une plateforme plus complète intégrant des fonctionnalités avancées telles que la gestion multi-établissements, les paiements électroniques, les programmes de fidélité, les commandes en ligne ou encore l'analyse intelligente des données de vente.

En définitive, ce projet constitue une réponse concrète aux besoins de digitalisation des débits de boissons et représente une base solide pour le développement d'une solution professionnelle, performante et adaptée aux exigences du terrain.

ANNEXES

Annexe A : Technologies utilisées

- **Frontend** : Flutter (Dart)
- **Backend** : Django (Python)
- **Base de données** : PostgreSQL
- **API** : REST
- **Sécurité** : JWT, Spring Security
- **Outils** : Git, GitHub, Postman, Figma, VS Code

Annexe B : Exemple de scénario utilisateur (vente POS)

1. L'utilisateur se connecte au système.
2. Il ouvre la caisse.
3. Il sélectionne les produits.
4. Le système calcule automatiquement le total.
5. Le client effectue le paiement.
6. Le système enregistre la vente.
7. Le stock est mis à jour automatiquement.
8. Un reçu est imprimé.

Annexe C: Captures d'écran